

**UNIVERSITY OF SCIENCE
FACULTY OF INFORMATION TECHNOLOGY**

Project Proposal

Introduction to Software Engineering

GROUP 5

Advisor: PhD. Tran Duy Hoang
MSc. Truong Phuoc Loc
MSc. Pham Hoang Hai



Software Engineering Department
Faculty of Information and Technology
University of Science

Table of Contents

1 Member Contribution Assessment	2
2 Preliminary Problem Statement	4
3 Proposed Solution	6
3.1 Software	6
3.1.1. Features	6
3.1.2. Software Architecture	9
3.2 Hardware	10
3.2.1. Server-Side Requirements	10
3.2.2. Client-Side Requirements	10
4 Development Plan	11
4.1 Requirements Analysis	11
4.2 Software Design	12
4.3 Implementation Plan	12
4.3.1. Implementation Environment Configuration	12
4.3.2. Detailed Implementation Steps	13
4.4 Comprehensive Testing Plan	14
4.4.1. Applied Testing Levels	14
4.4.2. User Acceptance Testing (UAT)	14
4.5 Deployment and Maintenance	15
4.5.1. System Deployment Plan	15
4.5.2. System Maintenance and Monitoring	16
5 Human Resources & Costing Plan	17
5.1 Personnel Structure	17
5.2 Costing Plan	18
6 Tools setup	19

1 Member Contribution Assessment

ID	Name	Contribution (%)	Signature
24127460	Nguyễn Hoàng Kim Ngân	100 %	
24127481	Nguyễn Đức Nhật Phát	100 %	
24127195	Nguyễn Gia Kiệt	100 %	
24127527	Ngô Tuyết Quyên	100 %	

2

Preliminary Problem Statement

Business Description

In a modern university environment, extracurricular activities play a vital role in the comprehensive development of students. However, the majority of clubs and event organizing committees at universities in Vietnam currently rely on manual management methods such as Google Forms, Excel spreadsheets, and chat groups on Zalo or Facebook. While simple, this approach reveals many serious limitations as the scale of events expands.

Specifically, the organizing committee lacks an automated mechanism to manage waitlists once events are full, and integrating participation data with training systems faces many constraints. For students, searching for and registering for events is fragmented, lacking a personalized recommendation system, causing many valuable activities to be missed.

Therefore, building a comprehensive web-based platform for event management and organization in a university environment is absolutely necessary. The system targets three main user groups: **Students**, **Organizers**, and **Administrators**, each with their own business flows and specific permissions.

For organizers, the system allows for the **creation, editing, and management of events** with full details such as time, location, maximum number of participants, and registration deadlines. Organizers can **upload banners**, set event status (draft/published/cancelled), send notifications to registrants, and monitor real-time participation statistics dashboards.

For students, the system provides an **event discovery** experience through search tools and filters by category, time, and popularity. Students can **register online**, save events of interest, and receive an automated **check-in QR code** upon successful registration. Participation history is recorded and accumulated into **activity points** displayed on a leaderboard, contributing to increased motivation to participate in extracurricular activities.

A highlight of the system is the integration of **artificial intelligence (AI)** to support both sides. The AI system automatically **suggests suitable events** based on each student's participation history and preferences, while also supporting organizers in **automatically generating event descriptions and promotional content**.

Operating Environment

The software will operate as a responsive web application:

- **Client-side:** Modern Web Browsers (Google Chrome, Mozilla Firefox, Safari, Microsoft Edge) supporting HTML5, CSS3, and ES6+. Mobile browsers must allow camera permissions for the QR code scanning feature.
- **Server-side:** A cloud hosting environment (e.g., AWS, Render, or Heroku) capable of executing asynchronous Python applications and hosting a relational database.

Design & Implementation Constraints

The system is built using the following technologies, each selected for a specific reason:

Technology	Reason
ReactJS + Tailwind CSS	Component-based UI speeds up development; Tailwind avoids writing custom CSS, saving time.
FastAPI (Python)	Fast to build REST APIs with automatic docs; async support fits real-time features like notifications.
Supabase (PostgreSQL)	Managed backend providing database, authentication, file storage, and realtime features, reducing setup work.

3

Proposed Solution

3.1 Software

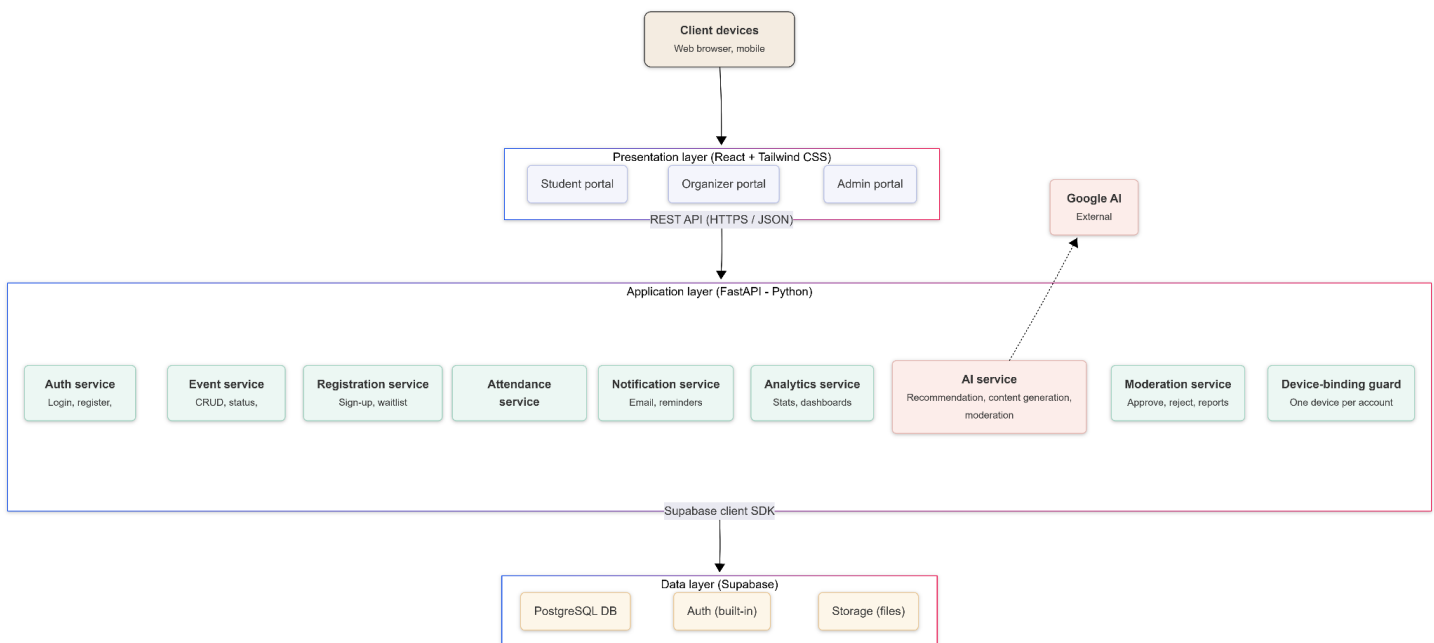
3.1.1. Features

User story	Requirement
As a new user, I want to register for an account using my university email and verify it.	User Registration & Verification
As a registered user, I want to securely log into and out of the system.	User Authentication
As a user who forgot their password, I want to request a password reset link via my registered email.	Password Recovery
As a user, I want to edit my personal details, upload an avatar, and change my password.	Profile Management
As an organizer, I want to create a new event with full details including title, time, location, maximum capacity, registration deadline and banner image so that students can be informed	Event Creation
As an organizer, I want to set and update the event status (Published, Cancelled) to manage its visibility to students.	Event Status Management
As an organizer, I want the system to trigger a notification to all registered participants whenever important event details are modified.	Event Notification

As an organizer, I want the system to generate a draft event description from basic input information.	AI Content Generation
As an organizer, I want to know the number of checked-in students in real-time so that I can stabilize seating in the hall.	Real-time Dashboard
As a student, I want to receive instant, automated answers to common event questions based on the event details.	Smart Q&A Moderator
As a student, I want to view my past event participation history and accumulated activity points	Activity Tracking
As a student, I want to search and filter events by category and time so that I can easily find relevant activities.	Event Search & Filtering
As a student, I want to register for an event online and receive immediate feedback on my registration status.	Event Registration
As a student, I want to save or bookmark events I am interested in to view or register later.	Event Bookmarking
As a student, I want the system to suggest events that match my interests, based on my participation history and accumulated activity points.	Personalized AI Recommendation
As a student, I want to receive notifications (urgent updates, new events) so that I can manage my schedule and earn training points.	Urgent: Instantly notify of any changes to time, location, or cancellations. New Event: Notify immediately upon open registration, displaying expected training points.

As an student, I want to check in quickly using my phone at the hall so that I can avoid standing in line to sign up.	QR Code Attendance
As an student, I want to have a manual code entry method when my phone camera fails so that I don't miss my attendance turn.	Manual Attendance Code Entry
As a student, I want to join the waiting list when an event reaches its participant limit and be automatically promoted to the registered participant list if a slot becomes available,so that I still have the opportunity to attend the event without having to register again.	
As an admin, I want to assign, modify, or revoke specific roles (Student, Organizer, Admin) for different accounts.	Role & Permission Management
As an admin, I want to approve or reject event requests submitted by organizers before they are published.	Event Approval
As an admin, I want to see registration and attendance statistics for each event.	Event Analytics Dashboard

3.1.2. Software Architecture



The Smart University Event Ecosystem utilizes a modern 3-tier architecture integrated with a Backend-as-a-Service (BaaS) model:

1. **Presentation Layer (Frontend):** Built with **ReactJS** and **Tailwind CSS**, this layer provides responsive, role-specific portals for Students, Organizers, and Admins. It handles user interactions, client-side validation, and in-browser hardware usage (e.g., QR scanning).
2. **Application Layer (Backend API):** Developed with **Python (FastAPI)**. It implements business rules that the BaaS layer cannot enforce natively — such as device-binding for secure attendance and approval workflows for event moderation. Within this layer, the **AI Service** is the component responsible for the three AI-driven features identified in the user stories: generating draft event descriptions from organizer input (AI Content Generation), ranking events for each student based on their history and activity points (Personalized AI Recommendation), and answering student questions about event details using the event's own data as context (Smart Q&A Moderator).

3. **Data Layer (BaaS):** Powered by **Supabase (PostgreSQL)**. It acts as a unified data hub managing the core relational database, built-in user authentication, and media storage (for event banners). It also leverages Supabase's Realtime engine to broadcast instant updates to the frontend (e.g., live attendance counters and immediate notifications).

3.2 Hardware

3.2.1. Server-Side Requirements

- **Application Server (Hosting the Python FastAPI Backend):**
 - **CPU:** Minimum 2 Cores (Recommended 4 Cores to efficiently handle concurrent HTTP requests and manage external integration flows with Google AI services).
 - **RAM:** Minimum 4 GB (Recommended 8 GB to guarantee smooth performance under heavy traffic and realtime data exchange via the Supabase client SDK).
 - **Storage:** 20 GB to 40 GB SSD (Mainly required for storing application code, API server logs, and background operational processes, as large media files and relational tables are handled externally by Supabase).
 - **Network:** High-speed, stable Internet connection with fixed public IP or domain mapping, configured with standard SSL for secure HTTPS REST API communication.

3.2.2. Client-Side Requirements

- **For Students (General Users):**
 - **End-user Devices:** A smartphone, tablet, or personal computer running a modern web browser (Google Chrome, Safari, Microsoft Edge) with active internet connectivity.
 - **Camera Module:** Smartphones must be equipped with a functional rear camera to utilize the in-browser QR scanning feature for event check-in operations.
- **For Event Organizers and Administrators:**
 - **Workstation Devices:** A personal laptop or desktop computer with standard input peripherals.
 - **Display Device:** A monitor with at least a standard Full HD resolution (1920 x 1080 pixels is recommended) to comfortably interact with the extensive admin control panels, real-time analytics dashboards, and statistical charts.

4 Development Plan

4.1 Requirements Analysis

Objective: To systematically identify and document the requirements for a centralized web-based platform that facilitates event discovery for students and promotes organizational activities for university-affiliated clubs and committees, while ensuring architectural readiness for future mobile application integration.

Key Activities:

- **Stakeholder Identification & Needs Assessment:**
 - *For Students:* Define requirements for a seamless event discovery experience, including advanced search filters (category, time, popularity) and personalized recommendations to maximize engagement in extracurricular activities.
 - *For Organizers (Clubs/Committees):* Establish functional requirements for self-service event management, including creating/editing event details (time, location, capacity, deadlines), media uploads (banners), and real-time monitoring of participation statistics via dashboards.
 - *For Administrators:* Define requirements for overseeing the entire ecosystem, including approval workflows for published events and role management for different user accounts.
- **Workflow Integration:** Map the lifecycle of an event within the system—from draft creation and administrative approval to publishing, student registration, and post-event attendance tracking.
- **System Feasibility & Constraint Analysis:**
 - Analyze the necessity of a responsive web design to ensure accessibility across desktop and mobile browsers for all university members.
 - Define non-functional requirements regarding system scalability to handle peak registration periods and secure data integration with university activity point systems.

- **Platform Compatibility & Scalability:** Establish an API-first approach during requirement gathering to decouple core business workflows from the presentation layer, ensuring that future native smartphone applications (iOS/Android) can be integrated without redefining core platform rules.
- **Requirement Documentation:** Translate these goals into a structured set of functional requirements, prioritizing features that directly support event promotion, student registration, and participation data management.

4.2 *Software Design*

Objective: Develop a robust technical foundation that supports scalability, security, and seamless user interaction, optimized for current web deployment and future multi-platform expansion.

Key Activities:

- **System Architecture Design:** Formalize the 3-tier architecture, ensuring clear separation of concerns between the ReactJS Presentation Layer, FastAPI Application Layer, and Supabase Data Layer. The application layer is strictly decoupled to act as a standalone backend engine, preparing the system for multi-client routing (Web and future Mobile apps).
- **Database Schema Design:** Develop the relational database schema in PostgreSQL, covering all entities including User accounts, Event metadata, Participation logs, and Attendance status.
- **API Specification:** Define RESTful API endpoints using a contract-based interface. By standardizing JSON-based request/response structures for microservices (Authentication, Event CRUD, Attendance tracking, and AI services), the backend serves as a universal data provider that can seamlessly connect to web clients today and native smartphone applications tomorrow.
- **UI/UX Blueprinting:** Design system wireframes and prototypes using Figma, ensuring a highly responsive web interface that accommodates desktop administration for organizers and effortless, mobile-browser-based QR check-in capabilities for students.

4.3 Implementation Plan

This section defines the process of configuring the infrastructure and implementing the Smart University Event Ecosystem system from the local development environment to serve the university.

4.3.1. Implementation Environment Configuration

- **Backend & Database Infrastructure:**
 - **FastAPI Backend:** Configured as an asynchronous RESTful API server that securely integrates with Supabase services (Authentication, PostgreSQL Database, and Storage) as well as AI services (Gemini API). The backend is responsible for authentication, business logic processing, database operations, and communication with external services.
 - **Supabase BaaS:** Used as a centralized data storage hub, including the PostgreSQL relational database, Supabase Authentication for user authentication and JWT management, Supabase Storage for storing event-related media files (e.g., banners and supporting images), and Realtime services for synchronizing attendance data.
- **Client-side Environment:**
 - **Responsive Web Application (ReactJS + Tailwind CSS):** Developed to operate smoothly on both desktop browsers (Web Admin for Organizers, Admins) and mobile browsers such as Google Chrome, Safari, Microsoft Edge (Web Client for Students).
 - **Mobile hardware requirements:** Students' devices must have a properly functioning rear camera, and the mobile browser must be granted Camera Permission by the user to execute the QR code scanning feature on the web.

4.3.2. Detailed Implementation Steps

- **Step 1: Configure Supabase Project and Database**
 - Create the Supabase project and configure database connection settings.
 - Define database models using SQLAlchemy ORM and generate the database schema through Alembic migrations.
 - Initialize the database schema, including tables for users, events, registrations, attendance logs, organizer requests, event images, and other supporting entities.
 - Configure primary key and foreign key constraints, indexes, and Row Level Security (RLS) policies to ensure secure data access.

- **Step 2: Configure Backend API (FastAPI)**
 - Set up the FastAPI project structure and configure secure environment variables, including SUPABASE_URL, SUPABASE_SERVICE_ROLE_KEY, SUPABASE_ANON_KEY, JWT_SECRET, and GEMINI_API_KEY. These credentials must never be exposed to the frontend.
 - Implement RESTful APIs for user management, event management, registrations, attendance tracking, organizer requests, and AI chatbot services.
 - Verify the connection status via the automated OpenAPI documentation (Swagger UI) at the server's /docs path.
- **Step 3: Develop Frontend Application**
 - Build responsive user interfaces using ReactJS and Tailwind CSS.
 - Integrate frontend components with FastAPI REST APIs to implement authentication, event browsing, registration, organizer management, QR code check-in, and administrative functions.
 - Ensure compatibility across modern desktop and mobile browsers.
- **Step 5: Integrate Frontend with Backend**
 - Connect the React frontend to FastAPI REST APIs using HTTP requests.
 - Implement secure authentication by attaching JWT tokens to protected API requests.
 - Validate end-to-end functionality, ensuring that frontend features correctly interact with backend services, database operations, and external integrations.

4.4 Comprehensive Testing Plan

The event management and QR attendance system will undergo a rigorous testing process across multiple levels to ensure stability, AI accuracy, and anti-fraud attendance tracking.

4.4.1. Applied Testing Levels

- **Unit Testing:** Focuses on verifying the accuracy of core logic functions developed by the team. Use the “pytest” framework (Python) to test functions for encoding/decoding QR code data strings, validating event time constraints, and enforcing device binding (limiting to one unique device).
- **Integration Testing:** Verifies the combination and data flow among three layers: ReactJS Interface → FastAPI API → Supabase Database Realtime Engine. The focus is to test the real-time data flow that automatically updates the number of checked-in students on the Organizer's Dashboard as soon as a student successfully scans the code on their phone.

- **Load Testing:** Use the Locust tool to simulate a bottleneck scenario at the event hall. Simulate 500 to 1000 students concurrently accessing the Mobile Web and sending QR attendance requests within a short timeframe (5-10 minutes before the start time) to measure server latency and optimize database queries.

4.4.2. User Acceptance Testing (UAT)

- **Pilot Testing:** Conduct a small-scale live system test during a small club event (approximately 30 - 50 participating students). This process helps test the responsiveness of the Web-based QR scanning feature across various smartphone models (iOS/Android) and verifies the stability of the 4G network in the lobby area.
- **Acceptance and Handover:** Collect direct feedback from Youth Union and Student Association officers (Organizers group) regarding the usability of the Event Management Dashboard and the feature to export attendance data to Excel files for future training points calculations. Finetune the final UI bugs before deploying the system officially.

4.5 *Deployment and Maintenance*

4.5.1. System Deployment Plan

The deployment strategy leverages modern cloud environments to separate the system layers, ensuring cost-efficiency, operational stability, and high performance during university event peaks.

- **Data Layer Deployment (Supabase):**
 - The production database schema, row-level security (RLS) policies, and storage buckets for event banners are transitioned from the testing environment to the official production instance on Supabase.
 - Database connection pooling is optimized to maintain stable, persistent connections with the application API layer during high-concurrency registration periods.
- **Application Layer Deployment (FastAPI Backend):**
 - The Python FastAPI codebase can be containerized using Docker to ensure environment consistency across systems.
 - The container is deployed onto cloud hosting platforms (such as Render, Railway, or university-hosted infrastructure).

- Secure Environment Variables (SUPABASE_URL, SUPABASE_SERVICE_ROLE_KEY, GEMINI_API_KEY, JWT_SECRET) are managed natively within the hosting platform's secure vault, preventing any source code exposure.
- **Presentation Layer Deployment (ReactJS Frontend):**
 - The ReactJS application is compiled into optimized, static production files.
 - These assets are deployed directly to a centralized cloud hosting service with servers located within Vietnam (such as a local datacenter or localized hosting provider). This ensures that all data traffic remains strictly inside the country, minimizing latency and providing maximum loading speed for university students accessing the platform from local cellular networks (4G/Wifi) at the campus.
 - The production domain is secured using an SSL certificate to guarantee that all web-based QR scanning and attendance features are strictly encrypted over HTTPS.

4.5.2. System Maintenance and Monitoring

- **Error Logging:** The FastAPI application utilizes basic logging middleware to record operational exceptions. This is critical for capturing and diagnosing integration failures, such as network timeouts when interacting with the external Gemini AI API.
- **Database Backups & Access Control:**
 - Manual and scheduled database backups are performed natively within the Supabase dashboard to safeguard core event records and student registration logs against accidental data loss.
 - Row-Level Security (RLS) policies on Supabase are strictly maintained to enforce data access boundaries, ensuring students can only view or modify their own participation data.

5

Human Resources & Costing Plan

5.1 Personnel Structure

Team Member	Project Role	Assigned Development Modules
Ngân	Project Manager & Fullstack Developer	• Authentication & User Management • Admin & Moderation Module
Kiệt	Full-stack Developer & AI Engineer	• Event Discovery & Recommendation • AI Content Generation & Categorization • Analytics & Dashboards
Phát	Fullstack Developer & UI/UX Lead	• Registration & Waiting List Workflow • QR Attendance System • Real-time Notification System • System UI/UX Prototyping (Figma)
Quyên	Full-stack Developer & QA	• Event CRUD & Configuration • Event Media Management • Hardware/Device testing

5.2 Costing Plan

Expense Category	Item / Service	Description	Estimated Cost
Infrastructure	Supabase (Free Tier)	Cloud-hosted PostgreSQL, Auth, and Storage.	\$0.00
Backend	FastAPI (Local)	Python backend running on local development machines.	\$0.00
Frontend	ReactJS (Local)	Node.js development server for local testing.	\$0.00
AI Integration	API Free Tiers	Trial credits for Gemini/OpenAI feature testing.	\$0.00
Dev Tools	GitHub (Educational)	Version control, private repositories, & CI/CD.	\$0.00
Total		Development and academic demonstration budget	\$0.00

6

Tools setup

To ensure effective teamwork and project management, our team utilizes the following tools:

- **Communication: Slack** (TA-provided workspace) for official discussions, announcements, and real-time updates.
- **Task Management: Jira** for tracking Agile workflows, sprints, and task assignments.
- **UI/UX Design: Figma** for collaborative wireframing and interface prototyping.
- **Assignment Submission: Moodle** for submitting course deliverables.
- **Version Control & Documentation: GitHub** for centralized repository management, strictly adhering to the required directory structure:
 - /src: Stores application source code.
 - /docs: Stores all project documentation, categorized into /management, /requirements, /analysis_and_design, and /test.
 - /pa: Stores Project Assignment submissions in dedicated subfolders.